

TAFSM File Format

June 10, 2024

1 File Formats

The following document contains explanations and formatting of various files.

2 FE File Format

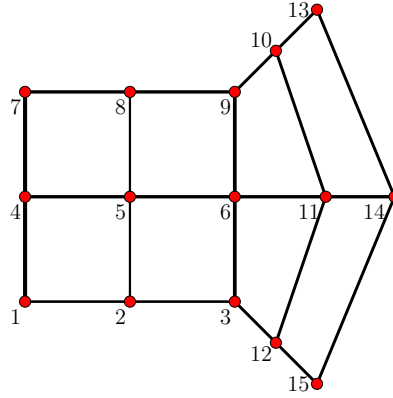


Figure 1: Example 2D FE mesh

2.1 Control File

2.1.1 mesh.cfg

Description: Describes mesh information and locates additional files. The endianness for binary files are determined by **endian**, which can be **big** or **little**, and **offset** is 0 or 1, which is the first node id. Number of spatial and parametric dimensions

are `nsd` and `npd`, respectively. The number of element-nodes is `nen`.

Format: Text

Type: ASCII

Size: –

Example:

```

1  [info]
2  version = 2.0
3  endian= big
4  nn = 15
5  ne = 8
6  offset = 1
7  [mesh]
8  nsd = 2
9  npd = 2
10 nen = 4
11 [file]
12 xyz = mxyz
13 ien = mien
14 rng = mrng

```

2.2 Node Points

2.2.1 xyz

Description: Contains nodal position information.

Format: Binary

Type: 8-byte floating-point

Size: $nn \times nsd \times 8$ bytes

Example:

```

1  -1.98166e-01 -1.98166e-01
2  0.00000e+00 -1.98166e-01
3  1.98166e-01 -1.98166e-01
4  -1.98166e-01 0.00000e+00
5  0.00000e+00 0.00000e+00
6  1.98166e-01 0.00000e+00
7  -1.98166e-01 1.98166e-01
8  0.00000e+00 1.98166e-01
9  1.98166e-01 1.98166e-01
10 3.96332e-01 3.22019e-01
11 4.45873e-01 0.00000e-01
12 3.96332e-01 -3.22019e-01
13 5.94498e-01 4.45873e-01
14 6.93581e-01 0.00000e-01
15 5.94498e-01 -4.45873e-01

```

2.3 Connectivity

2.3.1 ien

Description: Contains connectivity information. (Local node ids to global node ids.)

Format: Binary

Type: 4-byte integer

Size: $\mathbf{ne} \times \mathbf{nen} \times 4$ bytes

Example:

1	1	2	5	4
2	2	3	6	5
3	3	12	11	6
4	12	15	14	11
5	4	5	8	7
6	5	6	9	8
7	6	11	10	9
8	11	14	13	10

2.4 Boundary Numbers

2.4.1 rng

Description: Contains boundary numbers for elements where negative integers refer to interior boundaries between elements and positive integers refer to given domain boundaries.

Format: Binary

Type: 4-byte integer

Size: $\mathbf{nface} \times 4$ bytes

Here $\mathbf{nface}$ is number of element faces.

Example:

1	4	-1	-1	1
2	4	-1	-1	-1
3	4	-1	-1	-1
4	4	2	-1	-1
5	-1	-1	3	1
6	-1	-1	3	-1
7	-1	-1	3	-1
8	-1	2	3	-1

2.5 Properties

2.5.1 mat (optional)

Description: Contains material id where the same number represents the same material.

Format: Binary

Type: 4-byte integer

Size: $n_e \times 4$ bytes

Example:

1	57
2	57
3	56
4	56
5	57
6	57
7	56
8	56

3 NURBS File Format

This format is deprecated.

4 Bézier-Extraction File Format

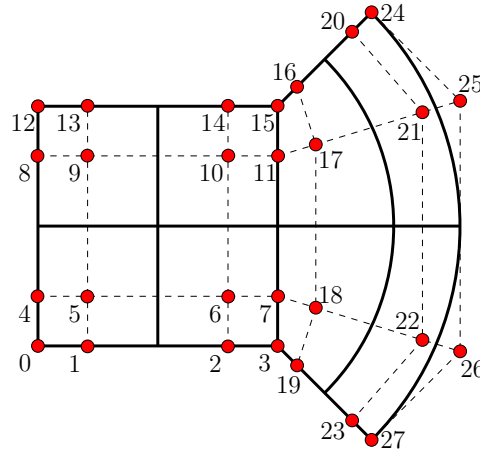


Figure 2: Example Bézier Extraction mesh

4.1 Control File

4.1.1 patch.cfg

Description: Describes mesh information and locates additional files. There are four versions, and they can be determined by `type`; `Bezier-extraction`, `Bezier-extraction2`, `Bezier-extraction3`, and `Bezier-extraction4`.

Format: Text

Type: ASCII

Size: –

Example:

```
1  [ info ]
2  type = Bezier-extraction
3  nn = 28
4  ne = 8
5  [ mesh ]
6  nsd = 2
7  max_npd = 2
8  [ file ]
9  cxyz = cxyz
10 ien = mien
11 ien_offset = ien_offset
12 rng = mrng
13 order = order
```

```

14 extraction-operator = C
15 extraction-operator_offset = C_offset

```

4.1.2 patch.cfg (Version 4)

Description: Similar to the previous `patch.cfg` but have additional information on `extraction-operator.index`. `type` is `Bezier-extraction4`.

Format: Text

Type: ASCII

Size: –

4.2 Control Points

4.2.1 cxyz

Description: Contains control points position information.

Format: Binary

Type: 8-byte floating-point

Size: $nn \times nsd \times 8$ bytes

Example:

```

1 -1.98166e-01 -1.98166e-01
2 -1.16083e-01 -1.98926e-01
3 1.16083e-01 -1.98926e-01
4 1.98166e-01 -1.98166e-01
5 -1.98926e-01 -1.16083e-01
6 -1.16528e-01 -1.16528e-01
7 1.16528e-01 -1.16528e-01
8 1.98926e-01 -1.16083e-01
9 -1.98926e-01 1.16083e-01
10 -1.16528e-01 1.16528e-01
11 1.16528e-01 1.16528e-01
12 1.98926e-01 1.16083e-01
13 -1.98166e-01 1.98166e-01
14 -1.16083e-01 1.98926e-01
15 1.16083e-01 1.98926e-01
16 1.98166e-01 1.98166e-01
17 2.30348e-01 2.30348e-01
18 2.61280e-01 1.34935e-01
19 2.61280e-01 -1.34935e-01
20 2.30348e-01 -2.30348e-01
21 3.21372e-01 3.21372e-01
22 4.37646e-01 1.88255e-01
23 4.37646e-01 -1.88255e-01
24 3.21372e-01 -3.21372e-01
25 3.53553e-01 3.53553e-01

```

```

26 5.00000e-01 2.07107e-01
27 5.00000e-01 -2.07107e-01
28 3.53553e-01 -3.53553e-01

```

4.3 Connectivity

4.3.1 ien

Description: Contains connectivity information. (Local node ids to global node ids).

Format: Binary

Type: 4-byte integer

Size: $ne \times nen \times 4$ bytes

Example:

1	0	1	2	4	5	6	8	9	10
2	1	2	3	5	6	7	9	10	11
3	4	5	6	8	9	10	12	13	14
4	5	6	7	9	10	11	13	14	15
5	15	11	7	16	17	18	20	21	22
6	11	7	3	17	18	19	21	22	23
7	16	17	18	20	21	22	24	25	26
8	17	18	19	21	22	23	25	26	27

4.3.2 ien_offset

Description: Indicates the start position of a new element in the **ien** list. The last entry should be the number of entries in **ien** file.

Format: Binary

Type: 4-byte integer

Size: $(ne + 1)$ bytes

Example:

1	0
2	9
3	18
4	27
5	36
6	45
7	54
8	63
9	72

4.4 Boundary Numbers

4.4.1 rng

Description: Contains boundary numbers for elements where negative integers refer to interior boundaries between elements and positive integers refer to given domain boundaries

Format: Binary

Type: 4-byte integer

Size: $ne \times 2^{\max_npd} \times 4$ bytes

Example:

1	1	-1	4	-1
2	-1	-1	4	-1
3	1	-1	-1	3
4	-1	-1	-1	3
5	3	-1	-1	-1
6	-1	4	-1	-1
7	3	-1	-1	2
8	-1	4	-1	2

4.5 Polynomial Order in Functions

4.5.1 order (Version 1)

Description: Order indicates the degree of the basis function for each direction for each element, so the number of control points is $\prod_{i=1}^{npd} (order_i + 1)$. Note that the column which has `order`= 0 reduce number of parametric dimensions.

Format: Binary

Type: 4-byte integer

Size: $ne \times \max_npd \times 4$ bytes

Example:

1	2	2
2	2	2
3	2	2
4	2	2
5	2	2
6	2	2
7	2	2
8	2	2

4.5.2 `order` (Version 3)

Description: Similar to the previous `order` but version 3 adds the first column indicating structured or unstructured. Structured means the same as before where the number of control points is `order + 1` for the remaining columns. Unstructured must be 0 or more than two directions and are based on T-spline shape functions. This is indicated by bit flags for only the unstructured direction. For example, a 3D mesh where the first and third directions are unstructured, the bit flag is $2^0 + 2^2 = 5$.

Format: Binary

Type: 4-byte integer

Size: $ne \times (\max_npd + 1) \times 4$ bytes

4.6 BézierExtraction Operator

4.6.1 `C` (Version 1)

Description: The Bézier extraction operator, `C` as defined for B-spline functions . When `npd` is less than `max_npd`, the order in that dimension should be set to 0 as explained in `ordersection`.

Format: Binary

Type: 8-byte floating-point

Size: $\sum_{e=1}^{ne} \left(\sum_{i=1}^{npd^e} (\text{order}_i^e + 1)^2 \right) \times 8$ bytes

Example:

1	1.00000e+00	0.00000e+00	0.00000e+00
2	0.00000e+00	1.00000e+00	5.00000e-01
3	0.00000e+00	0.00000e+00	5.00000e-01
4			
5	1.00000e+00	0.00000e+00	0.00000e+00
6	0.00000e+00	1.00000e+00	5.00000e-01
7	0.00000e+00	0.00000e+00	5.00000e-01
8			
9	5.00000e-01	0.00000e+00	0.00000e+00
10	5.00000e-01	1.00000e+00	0.00000e+00
11	0.00000e+00	0.00000e+00	1.00000e+00
12			
13	1.00000e+00	0.00000e+00	0.00000e+00
14	0.00000e+00	1.00000e+00	5.00000e-01
15	0.00000e+00	0.00000e+00	5.00000e-01
16			
17	1.00000e+00	0.00000e+00	0.00000e+00

18	0.00000e+00	1.00000e+00	5.00000e-01
19	0.00000e+00	0.00000e+00	5.00000e-01
20			
21	5.00000e-01	0.00000e+00	0.00000e+00
22	5.00000e-01	1.00000e+00	0.00000e+00
23	0.00000e+00	0.00000e+00	1.00000e+00
24			
25	5.00000e-01	0.00000e+00	0.00000e+00
26	5.00000e-01	1.00000e+00	0.00000e+00
27	0.00000e+00	0.00000e+00	1.00000e+00
28			
29	5.00000e-01	0.00000e+00	0.00000e+00
30	5.00000e-01	1.00000e+00	0.00000e+00
31	0.00000e+00	0.00000e+00	1.00000e+00
32			
33			
34	1.00000e+00	0.00000e+00	0.00000e+00
35	0.00000e+00	1.00000e+00	5.00000e-01
36	0.00000e+00	0.00000e+00	5.00000e-01
37			
38	1.00000e+00	0.00000e+00	0.00000e+00
39	0.00000e+00	1.00000e+00	5.00000e-01
40	0.00000e+00	0.00000e+00	5.00000e-01
41			
42	5.00000e-01	0.00000e+00	0.00000e+00
43	5.00000e-01	1.00000e+00	0.00000e+00
44	0.00000e+00	0.00000e+00	1.00000e+00
45			
46	1.00000e+00	0.00000e+00	0.00000e+00
47	0.00000e+00	1.00000e+00	5.00000e-01
48	0.00000e+00	0.00000e+00	5.00000e-01
49			
50	1.00000e+00	0.00000e+00	0.00000e+00
51	0.00000e+00	1.00000e+00	5.00000e-01
52	0.00000e+00	0.00000e+00	5.00000e-01
53			
54	5.00000e-01	0.00000e+00	0.00000e+00
55	5.00000e-01	1.00000e+00	0.00000e+00
56	0.00000e+00	0.00000e+00	1.00000e+00
57			
58	5.00000e-01	0.00000e+00	0.00000e+00
59	5.00000e-01	1.00000e+00	0.00000e+00
60	0.00000e+00	0.00000e+00	1.00000e+00
61			
62	5.00000e-01	0.00000e+00	0.00000e+00
63	5.00000e-01	1.00000e+00	0.00000e+00
64	0.00000e+00	0.00000e+00	1.00000e+00
65			

4.6.2 C (Version 2)

Description: The Bézier extraction operator, **C** as defined by mapping Bernstein polynomial basis functions to B-spline functions . When **npd** is less than **max_npd**, the order in that dimension should be set to 0. The file contains both structured

C s and an unstructured C for each element. Even if the unstructured directions are later directions, the unstructured C comes first. The row order is related to **ien**. The **ien** is unstructured direction first, and then the structured directions based on the parametric direction order.

Format: Binary

Type: 8-byte floating-point

Size: $\sum_{e=1}^{\mathbf{ne}} \mathbf{nen}^e \left(\prod_{i=1}^{\mathbf{npd}^e} (\mathbf{order}_i^e + 1) \right) \times 8 \text{ bytes}$

4.6.3 **C** (Version 3)

Description: The Bézier extraction operator, **C** consists of structured C s in each structured direction and an unstructured C . The C matrices are stored first for the unstructured directions and then for the structured directions.

Format: Binary

Type: 8-byte floating-point

Size:

4.6.4 **C** (Version 4)

Description: The Bézier extraction operator, **C** is stored in a generalized form of Version 1 format, and is suitable for both NURBS and T-spline without extraordinary points. Currently this format doesn't support multiple end point shape functions along one edge/face (i.e. having a C^0 continuous point in the middle of an edge). In this format, the unique extraction vectors in each parametric direction is stored, and the extraction operator for each basis function can be generated by taking tensor product of extraction operator in each parametric direction, similar to Version 1. Information on corresponding extraction vector for each basis function is stored in **C_id**, new file introduced in this version. All the extraction operator are padded with zero to the same length as the longest ones (direction with highest **order**). For NURBS mesh, Version 4 **C** is identical to Version 1 **C**

Format: Binary

Type: 8-byte floating-point

Size: $\sum_{e=1}^{\mathbf{ne}} (\sum_{i=1}^{\mathbf{npd}^e} (N_i^e)(\max(\mathbf{order}_i^e + 1))) \times 8$ bytes, where N_i^e represent the number of unique vectors in parametric direction i for element e

4.6.5 C_offset

Description: Indicates the start position of each element in the C list. The last entry should be the size of the C s entries.

Format: Binary

Type: 4-byte integer

Size: $(\mathbf{ne} + 1) \times 4$ bytes

Example:

1	0
2	18
3	36
4	54
5	72
6	90
7	108
8	126
9	144

4.6.6 C_id (Version 4)

Description: Indicate the indices of the extraction vectors corresponding to each basis function. The indices for each basis function is combined into a single 4-byte integer using the formula shown below. Offset for $\mathbf{C_id}$ is the same as $\mathbf{ien_offset}$, and is not stored separately to same memory. The value is

$$\mathbf{ind} = \sum_{i=1}^{\mathbf{npd}^e} \mathbf{ind}_i (N_i^e)^{(i-1)}$$

where $N^e = \sum_{i=1}^{\mathbf{npd}^e} N_i^e$. Note that we do not store N_i^e anywhere and $\mathbf{ind}_i$ is counting from the first parametric dimension to the last parametric dimension.

Remark 1 *Although, the stride N^e is the entire size rather than N_i^e . This is to not provide N_i^e in the format. The extraction-operators are placed in the same order of the parametric dimensions. This restriction was added to simply the implementation based on this data structure.*

Format: Binary

Type: 4-byte integer

Size: $n_e \times n_{en} \times 4$ bytes

Example:

1	18
2	19
3	20
4	24
5	25
6	26
7	30
8	31
9	32

The example is for a quadratic NURBS with 3×3 control points, which can be expressed with Version 1. The numbers are based on $18 = 0 \times 6^0 + 3 \times 6^1$, $19 = 1 \times 6^0 + 3 \times 6^1$, $24 = 0 \times 6^0 + 4 \times 6^1$, $30 = 0 \times 6^0 + 5 \times 6^1$, and $32 = 2 \times 6^0 + 5 \times 6^1$.

4.7 Node to Elements (optional)

4.7.1 n_{2e}

Description: Contains element `ids` for each node.

Format: Binary

Type: 4-byte integer

Size: $n \times n_{2e} \times 4$ bytes

Example:

1	0
2	0 1
3	0 1
4	1 5
5	0 2
6	0 1 2 3
7	0 1 2 3
8	1 3 4 5
9	0 2
10	0 1 2 3
11	0 1 2 3
12	1 3 4 5
13	2
14	2 3
15	2 3
16	3 4
17	4 6
18	4 5 6 7
19	4 5 6 7

20	5 7
21	4 6
22	4 5 6 7
23	4 5 6 7
24	5 7
25	6
26	6 7
27	6 7
28	7

The size `n_n2e` is the total size of `ien`.

4.7.2 n2e_offset

Description: Indicates the start position of each node `n2e` list. The last entry is the `n_n2e`.

Format: Binary

Type: 4-byte integer

Size: $(nn + 1)$ bytes

Example:

1	0
2	1
3	3
4	5
5	7
6	9
7	13
8	17
9	21
10	23
11	27
12	31
13	35
14	36
15	38
16	40
17	42
18	44
19	48
20	52
21	54
22	56
23	60
24	64
25	66
26	67
27	69
28	71
29	72

Format: Binary

Type: 4-byte integer

Size: $ne \times 4$ bytes

4.8 Control Point Coordinates (optional)

4.8.1 cpc

Description: Contains coordinates information for each element in each control point, compared with the element coordinates in each element.

Format: Binary

Type: 4-byte integer

Size: $n_n2e \times 4$ bytes

The size n_n2e is the total size of ien .

It is an additional information to $n2e$, and the same size of that. The offset file $n2e_offset$ is required to read the data. We note that the coordinate system for a control point is arbitrary and therefore that can be decided based on the program. For example, you can use the smallest element coordinates as the point coordinate system. We use two numbers, but compressed them into 4-byte integer value. For first npd bits are $dbit$, which indicates whether the direction is the same “0” or different “1.” Note that first bit is the first direction. After that bits we put a number that indicates the order of the axes, which has $npd!$ patterns. For that, we prepare the numbers that indicates the control point coordinates. For example, $npd = 3$, “123” is the case without changing the coordinates, and “132” indicates that the second and third axes are changed. Then the patterns are serialized as the order such that the lower axes will have earlier id. See Table 1 as an example. Overall an entry of the cpc is $axes \ll npd \mid dbit$, and we can obtain $axes = cpc \gg npd$.

4.9 NURBS Weights

4.9.1 weights (optional)

Description: The weight associated with each control point.

Format: Binary

Type: 8-byte floating-point

Table 1: **axes** for $\text{npd} = 3$. In an element, we find the direction compared with the control point coordinates. The first number is the first axis, and the second and third numbers are in the second and third axes

axes	Axes
0	123
1	132
2	212
3	221
4	312
5	321

Size: $\text{nn} \times 4$ bytes

Example:

1	1.00000e+00
2	8.53553e-01
3	8.53553e-01
4	1.00000e+00
5	8.53553e-01
6	7.28553e-01
7	7.28553e-01
8	8.53553e-01
9	8.53553e-01
10	7.28553e-01
11	7.28553e-01
12	8.53553e-01
13	1.00000e+00
14	8.53553e-01
15	8.53553e-01
16	1.00000e+00
17	8.53553e-01
18	7.28553e-01
19	7.28553e-01
20	8.53553e-01
21	8.53553e-01
22	7.28553e-01
23	7.28553e-01
24	8.53553e-01
25	1.00000e+00
26	8.53553e-01
27	8.53553e-01
28	1.00000e+00